

👉👉 Two Pointers + Sliding Window — Complete Beginner Notes

Goal: Master 5 patterns to solve all 5 NeetCode Two Pointers + all 6 Sliding Window problems. Sliding Window IS two pointers — both use a `left` and `right` on the same array/string. Notes are fully self-sufficient — no lecture needed.

📦 Table of Contents

PART A — Fundamentals

1. What is the Two Pointer Technique?
2. What is a Sliding Window?
3. Fixed vs Variable Window — The Core Difference
4. The Two Master Templates (Striver L1)
5. The "Exactly K = At Most K – At Most (K-1)" Trick

PART B — 5 Core Patterns 6. Pattern 1 — Opposite Ends Two Pointers 7. Pattern 2 — Fixed Size Sliding Window 8. Pattern 3 — Variable Window (Maximise Length) 9. Pattern 4 — Variable Window with Frequency Map (Minimise Length) 10. Pattern 5 — Monotonic Deque (Window Maximum) 11. How to Identify the Pattern (Decision Tree) 12. Problem → Pattern Mapping 13. Complexity Cheat Sheet 14. Common Mistakes to Avoid

PART A — Fundamentals

1. What is the Two Pointer Technique?

Two Pointers = use **two index variables** to avoid nested loops, reducing $O(n^2) \rightarrow O(n)$.

Two Flavours

```
FLAVOUR A — Opposite Ends (converging)
  left starts at 0, right starts at n-1.
  They move TOWARD each other until they meet.
```

```
[a, b, c, d, e, f]
  ↑           ↑
left         right
```

Use for: palindrome check, two sum on sorted array,
3Sum, container with most water, trapping rain water.

FLAVOUR B – Same Direction (sliding window)

left and right both start at 0.
right expands the window. left shrinks it.

[a, b, c, d, e, f]

↑ ↑

left right → right moves right every step
→ left moves right when window is invalid

Use for: substring problems, subarray problems with a condition.

Why Two Pointers Works

Naive approach for sorted array [1,3,5,7,9], target=10:

Fix every pair (i,j): $O(n^2)$
[1+3, 1+5, 1+7, 1+9, 3+5, 3+7, ...]

Two pointer insight: the array is SORTED.

left=1, right=9. Sum=10 → done!

If sum > target → right must decrease → right--

If sum < target → left must increase → left++

Each pointer moves at most n steps → $O(n)$ total.

2. What is a Sliding Window?

A **window** is a contiguous subarray or substring defined by two pointers **left** and **right**.

Array/String: [a, b, c, d, e, f, g]

Window [1..4]: [b, c, d, e]

↑ ↑

left right

Window size = right - left + 1

Why "sliding"? The window moves (slides) to the right as we iterate. Instead of recomputing from scratch for every window, we ADD the new right element and REMOVE the old left element — $O(1)$ update instead of $O(k)$.

Fixed window of size 3 sliding over [1, 2, 3, 4, 5]:

[1,2,3] → sum=6

[2,3,4] → sum=6+4-1=9 ← ADD 4, REMOVE 1 (O(1) update!)

[3,4,5] → sum=9+5-2=12

3. Fixed vs Variable Window

FIXED WINDOW: window size k is GIVEN.

"Find max sum of any subarray of size k"

→ right - left + 1 always = k

→ when right-left+1 > k: shrink from left (left++)

VARIABLE WINDOW: window size changes based on a CONDITION.

"Find LONGEST substring with at most 2 distinct characters"

→ expand right as long as condition is satisfied

→ when condition is violated: shrink from left (left++)

→ track max window size seen

"Find SHORTEST substring containing all target characters"

→ expand right until condition is satisfied

→ when condition is satisfied: try to shrink from left (left++)

→ track min window size seen

4. The Two Master Templates

Striver L1 defines these two templates. **Everything else is a variation.**

Template A — Maximise Window (shrink only when INVALID)

Used for: "Longest substring / subarray satisfying condition"

Strategy: Keep expanding. Only shrink when you MUST (condition broken).

java

```
// TEMPLATE A: Find LONGEST window satisfying condition
int left = 0;
int maxLen = 0;
// (some state variable: freq map, count, etc.)

for (int right = 0; right < n; right++) {
    // EXPAND: add arr[right] to window
    // update state...

    // SHRINK: while window is INVALID, move left
    while (/* window is invalid */) {
        // remove arr[left] from window
        // update state...
        left++;
    }

    // Window [left..right] is now valid – update answer
    maxLen = Math.max(maxLen, right - left + 1);
}
return maxLen;
```

Template B — Minimise Window (shrink while VALID)

Used for: "Shortest substring containing all characters / sum $\geq$ K"
Strategy: Expand until valid. Then shrink from left AS LONG AS still valid.

java

```
// TEMPLATE B: Find SHORTEST window satisfying condition
int left = 0;
int minLen = Integer.MAX_VALUE;
// (some state variable)

for (int right = 0; right < n; right++) {
    // EXPAND: add arr[right] to window
    // update state...

    // SHRINK: while window is VALID, try to minimise it
    while (/* window is valid */) {
        minLen = Math.min(minLen, right - left + 1); // update answer INSIDE
        // remove arr[left] from window
        // update state...
        left++;
    }
}
return minLen == Integer.MAX_VALUE ? 0 : minLen;
```

Key Difference:

	Template A (max window)	Template B (min window)
Answer updated	OUTSIDE the while loop	INSIDE the while loop
Shrink when	Window is INVALID	Window is VALID (try to shrink)
Goal	LARGEST valid window	SMALLEST valid window

5. The "Exactly K" Trick

Striver L9/L10/L11 teaches this. Very useful for "count subarrays with EXACTLY K of something."

DIRECT "exactly K" windows are hard to maintain because adding one element might flip from valid to invalid instantly.

BUT: atMost(K) is easy – just shrink when count > K.

KEY IDENTITY:

$$\text{count(exactly K)} = \text{count(at most K)} - \text{count(at most K-1)}$$

Example: "Count subarrays with exactly 2 odd numbers"
= count(at most 2 odd) - count(at most 1 odd)

java

```
// Generic atMost(k) helper
int atMost(int[] nums, int k) {
    int left = 0, count = 0, result = 0;
    for (int right = 0; right < nums.length; right++) {
        // expand: add nums[right] to window
        if (/* nums[right] contributes */) count++;

        // shrink: while count > k, remove from left
        while (count > k) {
            if (/* nums[left] contributed */) count--;
            left++;
        }

        // every window ending at right with left..right is valid
        result += right - left + 1; // count all valid subarrays ending at r
    }
    return result;
}

int exactlyK(int[] nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}
```

PART B — 5 Core Patterns

PATTERN 1 — Opposite Ends Two Pointers

When to Use

- Array or string is **sorted** (or can be treated symmetrically)
- Need to find a PAIR or check a property from both ends
- Keywords: "palindrome", "two sum sorted", "3sum", "container", "trap water"

The Core Logic

```
left = 0, right = n-1
```

At each step, make a decision based on the values at both pointers:

sum too big → right-- (decrease the larger side)

sum too small → left++ (increase the smaller side)

equal → record answer, then move both

Stop when left >= right.

Example A — Valid Palindrome

Problem: Is the string a palindrome after removing non-alphanumeric characters and ignoring case?

```
"A man, a plan, a canal: Panama" → true
```

```
"race a car" → false
```

```
java
```

```

class Solution {
    public boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;

        while (left < right) {
            // Skip non-alphanumeric from left
            while (left < right && !Character.isLetterOrDigit(s.charAt(left)))
                left++;
            // Skip non-alphanumeric from right
            while (left < right && !Character.isLetterOrDigit(s.charAt(right)))
                right--;

            // Compare (case-insensitive)
            if (Character.toLowerCase(s.charAt(left)) !=
                Character.toLowerCase(s.charAt(right)))
                return false;

            left++;
            right--;
        }
        return true;
    }
}
// Time: O(n) | Space: O(1)

```

Trace for "A man":

"A man" → after lowercasing relevant chars: a, m, a, n
 left=0('A'), right=4('n'): skip spaces/commas
 Compare 'a' vs 'n' → not equal → NOT palindrome

For "racecar":

left=0('r'), right=6('r'): match → left=1, right=5
 left=1('a'), right=5('a'): match → left=2, right=4
 left=2('c'), right=4('c'): match → left=3, right=3
 left >= right → STOP → return true ✅

Example B — Two Sum II (Input Array Is Sorted)

Problem: Sorted array, find two numbers that add up to target. Return 1-indexed positions.

java

```
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int left = 0, right = numbers.length - 1;

        while (left < right) {
            int sum = numbers[left] + numbers[right];

            if (sum == target) {
                return new int[]{left + 1, right + 1}; // 1-indexed
            } else if (sum < target) {
                left++; // need larger sum → move left right
            } else {
                right--; // need smaller sum → move right left
            }
        }

        return new int[]{}; // guaranteed to have solution
    }
}

// Time: O(n) | Space: O(1)
```

Why this works on sorted arrays:

```
[2, 7, 11, 15], target=9
left=2, right=15: sum=17 > 9 → right-- (15 is too big)
left=2, right=11: sum=13 > 9 → right--
left=2, right=7: sum=9 = 9 → FOUND [1,2] ✓
```

If we used a HashMap instead, we'd need $O(n)$ extra space.
Two pointer uses $O(1)$ space because the array is already sorted!

Example C — 3Sum

Problem: Find all unique triplets that sum to zero.

```
nums = [-1, 0, 1, 2, -1, -4]
Answer: [[-1,-1,2],[-1,0,1]]
```

Strategy:

Fix one element (i), then use two pointers on the REST of the sorted array. For each i , find pairs in $\text{nums}[i+1..n-1]$ that sum to $-\text{nums}[i]$.

MUST sort first (for two pointers + duplicate skipping).

Skip duplicates:

For i : if $\text{nums}[i] == \text{nums}[i-1] \rightarrow$ skip (same triplet)

For left/right: after finding a triplet, skip duplicates

java

```

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums); // MUST sort first
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < nums.length - 2; i++) {
            // Skip duplicate values for the fixed element
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            // Early exit: smallest possible sum > 0
            if (nums[i] > 0) break;

            int left = i + 1, right = nums.length - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left], nums[right]));

                    // Skip duplicates for left and right
                    while (left < right && nums[left] == nums[left + 1]) left++;
                    while (left < right && nums[right] == nums[right - 1]) right--;

                    left++;
                    right--;
                } else if (sum < 0) {
                    left++; // need larger sum
                } else {
                    right--; // need smaller sum
                }
            }
        }
        return result;
    }
}

// Time:  $O(n^2)$  -  $O(n \log n)$  sort +  $O(n^2)$  for all pairs
// Space:  $O(1)$  extra (output doesn't count)

```

Trace for [-4,-1,-1,0,1,2] (sorted):

```

i=0 (nums[i]=-4): left=1,right=5 → sum=-4+(-1)+2=-3<0→left++
                left=2,right=5 → sum=-4+(-1)+2=-3<0→left++
                left=3,right=5 → sum=-4+0+2=-2<0→left++
                left=4,right=5 → sum=-4+1+2=-1<0→left++
                left=5>right=5 → stop
i=1 (nums[i]=-1): left=2,right=5 → sum=-1+(-1)+2=0→add[-1,-1,2]
                skip dups→left=3,right=4 → sum=-1+0+1=0→add[-1,0,1]
                left=4>right=3? → stop
i=2: nums[2]=-1==nums[1]=-1 → SKIP (duplicate)
i=3 (nums[i]=0): left=4,right=5 → sum=0+1+2=3>0 → right--
                left>right → stop
i=4: nums[i]=1>0 → BREAK

```

Result: `[[-1,-1,2],[-1,0,1]]` 

Example D — Container With Most Water

Problem: Given heights array, find two lines that form a container holding the most water.

```

heights = [1, 8, 6, 2, 5, 4, 8, 3, 7]
Answer: 49 (lines at index 1 and 8, height=min(8,7)=7, width=8-1=7 →
area=49)

```

Key Insight:

Area = $\min(\text{height}[\text{left}], \text{height}[\text{right}]) \times (\text{right} - \text{left})$

Two pointer strategy:

Always move the SHORTER line inward.

Why? Moving the taller line can only DECREASE or keep the same width,
and the height is still limited by the shorter – can only get worse.
Moving the shorter line has POTENTIAL to find a taller line.

java

```

class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1;
        int maxWater = 0;

        while (left < right) {
            int water = Math.min(height[left], height[right]) * (right - left);
            maxWater = Math.max(maxWater, water);

            // Move the SHORTER line inward
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return maxWater;
    }
}
// Time: O(n) | Space: O(1)

```

Trace for [1,8,6,2,5,4,8,3,7]:

```

left=0(h=1), right=8(h=7): water=min(1,7)×8=8, h[left]<h[right]→left++
left=1(h=8), right=8(h=7): water=min(8,7)×7=49, h[right]<h[left]→right--
left=1(h=8), right=7(h=3): water=min(8,3)×6=18, right--
left=1(h=8), right=6(h=8): water=min(8,8)×5=40, right-- (equal: either
works)
...continues, maxWater stays at 49

```

Return 49 

Example E — Trapping Rain Water (Hard)

Problem: Given heights of walls, compute total water trapped after rain.

```
heights = [0,1,0,2,1,0,1,3,2,1,2,1]
```

Answer: 6

```

X           X
X  X  X X X X
X X X X X X X X X X X X
                ↑ water fills the valleys
```

Key Insight:

Water at position $i = \min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$

$\text{maxLeft}[i]$ = tallest wall to the LEFT of i (including i)

$\text{maxRight}[i]$ = tallest wall to the RIGHT of i (including i)

Water is bounded by the SHORTER of the two walls.

If shorter wall $>$ height[i] $\rightarrow$ water = shortWall - height[i]

If shorter wall $\leq$ height[i] $\rightarrow$ no water (position i is a wall itself)

Two Pointer approach ($O(1)$ space):

We don't need to precompute $\text{maxLeft}[]$ and $\text{maxRight}[]$ arrays.

Use two pointers. Maintain maxLeft and maxRight as running variables.

Key observation:

If $\text{maxLeft} \leq \text{maxRight}$:

water at left = $\text{maxLeft} - \text{height}[\text{left}]$ (right wall is taller, left is the bottleneck)

process left pointer, left++

Else:

water at right = $\text{maxRight} - \text{height}[\text{right}]$

process right pointer, right--

java

```

class Solution {
    public int trap(int[] height) {
        int left = 0, right = height.length - 1;
        int maxLeft = 0, maxRight = 0;
        int totalWater = 0;

        while (left < right) {
            if (height[left] <= height[right]) {
                // Left side is the bottleneck
                if (height[left] >= maxLeft) {
                    maxLeft = height[left]; // update max, no water here
                } else {
                    totalWater += maxLeft - height[left]; // trap water
                }
                left++;
            } else {
                // Right side is the bottleneck
                if (height[right] >= maxRight) {
                    maxRight = height[right]; // update max, no water here
                } else {
                    totalWater += maxRight - height[right]; // trap water
                }
                right--;
            }
        }
        return totalWater;
    }
}
// Time: O(n) | Space: O(1)

```

Trace for [0,1,0,2,1,0,1,3,2,1,2,1]:

```
left=0(h=0), right=11(h=1), maxL=0, maxR=0:  
  h[left]=0 ≤ h[right]=1 → left side  
  h[0]=0 ≥ maxL=0 → maxL=0, no water. left=1
```

```
left=1(h=1), right=11(h=1), maxL=0:  
  h[1]=1 ≥ maxL=0 → maxL=1. left=2
```

```
left=2(h=0), right=11(h=1), maxL=1:  
  h[2]=0 < maxL=1 → water += 1-0 = 1. left=3
```

```
left=3(h=2), right=11(h=1):  
  h[left]=2 > h[right]=1 → right side  
  h[11]=1 ≥ maxR=0 → maxR=1. right=10
```

...continues...

Total = 6 

PATTERN 2 — Fixed Size Sliding Window

When to Use

- Window size K is explicitly given
- "Max/min/sum of every subarray of size K"
- "Does a permutation of pattern exist as a substring?" (pattern length = window size)
- Keywords: "subarray of size K", "permutation in string", "anagram in string"

The Core Logic

Maintain a window of exactly size K.

When window size exceeds K: remove leftmost element (left++).

```
for (right = 0; right < n; right++) {  
  add arr[right] to window  
  
  if (right - left + 1 > k): // window too large  
    remove arr[left] from window  
    left++  
  
  if (right - left + 1 == k): // full window
```



```
update answer
```

```
}
```

Example A — Permutation in String

Problem: Does `s2` contain a permutation of `s1` as a substring?

```
s1 = "ab", s2 = "eidbaooo"
```

```
Answer: true ("ba" is a permutation of "ab" and appears in s2)
```

Key Insight:

A permutation of `s1` = any string with same character frequencies as `s1`.

→ Slide a window of size `s1.length()` over `s2`.

→ At each window: does window have same freq map as `s1`?

→ Instead of rebuilding freq map: maintain running window freq.

When a character freq in window matches `s1`'s freq: "matched" counter.

Window is valid when ALL `s1.length()` characters are matched.

```
java
```

```

class Solution {
    public boolean checkInclusion(String s1, String s2) {
        if (s1.length() > s2.length()) return false;

        int[] need = new int[26];    // freq needed from s1
        int[] have = new int[26];    // freq in current window

        for (char c : s1.toCharArray()) need[c - 'a']++;

        int matched = 0;             // how many chars have correct freq
        int required = 0;            // how many distinct chars in s1 need mat
        for (int f : need) if (f > 0) required++;

        int left = 0;
        for (int right = 0; right < s2.length(); right++) {
            // EXPAND: add s2[right] to window
            int c = s2.charAt(right) - 'a';
            have[c]++;
            if (need[c] > 0 && have[c] == need[c]) matched++; // exact match

            // SHRINK: if window too large, remove s2[left]
            if (right - left + 1 > s1.length()) {
                int l = s2.charAt(left) - 'a';
                if (need[l] > 0 && have[l] == need[l]) matched--; // lost a
                have[l]--;
                left++;
            }

            // CHECK: all required chars matched?
            if (matched == required) return true;
        }
        return false;
    }
}
// Time: O(n) | Space: O(1) – fixed 26-char arrays

```

Simpler approach — compare freq arrays directly:

```
java
```

```

class Solution {
    public boolean checkInclusion(String s1, String s2) {
        if (s1.length() > s2.length()) return false;
        int[] need = new int[26], window = new int[26];

        for (char c : s1.toCharArray()) need[c - 'a']++;

        for (int i = 0; i < s2.length(); i++) {
            window[s2.charAt(i) - 'a']++;           // add right

            if (i >= s1.length())
                window[s2.charAt(i - s1.length()) - 'a']--; // remove left

            if (Arrays.equals(need, window)) return true;    // check
        }
        return false;
    }
}
// Time: O(26n) = O(n) | Space: O(1)
// Simpler but Arrays.equals on 26-element array is O(26) = O(1)

```

Example B — Maximum Points from Cards (Striver L2)

Problem: Pick exactly K cards from either end of a row. Maximise total points.

```

cardPoints = [1, 2, 3, 4, 5, 6, 1], k = 3
Answer: 12 (pick from right: 6+1+5 = 12... wait, pick 1,6,1? No.)
          Pick 5+6+1=12 from right end ✓

```

Key Insight — Sliding Window Inversion:

```

Taking k cards from the ends = leaving (n-k) cards in the MIDDLE untouched.
→ Find the minimum sum subarray of size (n-k).
→ Answer = totalSum - minSubarraySum(n-k).

```

java

```

class Solution {
    public int maxScore(int[] cardPoints, int k) {
        int n = cardPoints.length;
        int windowSize = n - k;    // size of the middle part to EXCLUDE

        // Edge case: take all cards
        if (windowSize == 0) {
            int sum = 0;
            for (int p : cardPoints) sum += p;
            return sum;
        }

        // Find minimum sum window of size (n-k)
        int windowSum = 0;
        for (int i = 0; i < windowSize; i++) windowSum += cardPoints[i];

        int minWindowSum = windowSum;
        for (int i = windowSize; i < n; i++) {
            windowSum += cardPoints[i] - cardPoints[i - windowSize]; // sliding window
            minWindowSum = Math.min(minWindowSum, windowSum);
        }

        int totalSum = 0;
        for (int p : cardPoints) totalSum += p;

        return totalSum - minWindowSum;
    }
}
// Time: O(n) | Space: O(1)

```

PATTERN 3 — Variable Window (Maximise Length)

When to Use

- Find the LONGEST subarray/substring satisfying a condition
- The condition involves: distinct characters $\leq K$, at most K replacements, no repeating chars
- **Template A** — expand freely, shrink only when condition broken

The Shrink Condition

The key to each problem is defining WHEN the window becomes invalid. Once you know the "invalid" condition, the template is always the same.

Problem	Invalid When
Longest substring no repeating	any char appears > 1 time
Longest at most K distinct chars	distinct chars > K
Fruit into baskets (at most 2 types)	distinct fruit types > 2
Longest repeating char replacement	(window size - max freq) > k

Example A — Longest Substring Without Repeating Characters

Problem: Find the length of the longest substring with no repeating characters.

s = "abcabcbb"
Answer: 3 ("abc")

```
java
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> lastIndex = new HashMap<>(); // char → last
        int left = 0, maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);

            // If c was seen AND is inside current window → shrink left past
            if (lastIndex.containsKey(c) && lastIndex.get(c) >= left) {
                left = lastIndex.get(c) + 1; // jump left past the duplicate
            }

            lastIndex.put(c, right); // update last seen index
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
// Time: O(n) | Space: O(min(n, 26)) – at most 26 unique chars
```

Trace for "abcabcb":

```
r=0('a'): left=0. window=[a]. maxLen=1. lastIdx={a:0}
r=1('b'): left=0. window=[a,b]. maxLen=2. lastIdx={a:0,b:1}
r=2('c'): left=0. window=[a,b,c]. maxLen=3. lastIdx={...,c:2}
r=3('a'): 'a' last at 0 ≥ left=0 → left=0+1=1.
           window=[b,c,a]. maxLen=3. lastIdx={a:3,...}
r=4('b'): 'b' last at 1 ≥ left=1 → left=1+1=2.
           window=[c,a,b]. maxLen=3. lastIdx={b:4,...}
r=5('c'): 'c' last at 2 ≥ left=2 → left=2+1=3.
           window=[a,b,c]. maxLen=3.
r=6('b'): 'b' last at 4 ≥ left=3 → left=4+1=5.
           window=[c,b]. maxLen=3.
r=7('b'): 'b' last at 6 ≥ left=5 → left=6+1=7.
           window=[b]. maxLen=3.
```

Return 3 

Example B — Longest Repeating Character Replacement

Problem: Given string (s) and integer (k) , find the longest substring where you can replace at most (k) characters to make all characters the same.

```
s = "AABABBA", k = 1
Answer: 4 (replace one B in "AABA" or "ABBA" to get "AAAA" or
"ABBA" → "AAAA")
```

Key Insight:

In any window, to make all characters the same, we keep the MOST FREQUENT character

and replace ALL others. Cost = windowSize - maxFreq.

If (windowSize - maxFreq) $\leq$ k $\rightarrow$ window is VALID (can make uniform with $\leq$ k replacements)

If (windowSize - maxFreq) > k $\rightarrow$ window is INVALID $\rightarrow$ shrink left

Track maxFreq in current window (we only need the max, not all freqs).

IMPORTANT TRICK: We never decrease maxFreq.

Why? We only care about LONGER windows. A shorter valid window won't beat our answer.

If the window shrinks, we don't update maxFreq downward – this might keep a slightly "stale" maxFreq, but it means we only expand when we can beat our best answer.

java

```
class Solution {
    public int characterReplacement(String s, int k) {
        int[] freq = new int[26];
        int left = 0, maxFreq = 0, maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            freq[s.charAt(right) - 'A']++;
            maxFreq = Math.max(maxFreq, freq[s.charAt(right) - 'A']);

            // INVALID: cost of replacements exceeds k
            while ((right - left + 1) - maxFreq > k) {
                freq[s.charAt(left) - 'A']--;
                left++;
                // Note: maxFreq not updated here (intentional – see explanation)
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
// Time: O(n) | Space: O(1)
```

Trace for "AABABBA", k=1:

```
r=0('A'): freq[A]=1, maxFreq=1. window=A(1). 1-1=0≤1 valid. maxLen=1
r=1('A'): freq[A]=2, maxFreq=2. window=AA(2). 2-2=0≤1 valid. maxLen=2
r=2('B'): freq[B]=1, maxFreq=2. window=AAB(3). 3-2=1≤1 valid. maxLen=3
r=3('A'): freq[A]=3, maxFreq=3. window=AABA(4). 4-3=1≤1 valid. maxLen=4
r=4('B'): freq[B]=2, maxFreq=3. window=AABAB(5). 5-3=2>1 INVALID
    → freq[A]--, left=1. freq[A]=2. window=ABAB(4). 4-3=1≤1 valid.
r=5('B'): freq[B]=3, maxFreq=3. window=ABABB(5). 5-3=2>1 INVALID
    → freq[A]--, left=2. freq[A]=1. window=BABB(4). 4-3=1≤1 valid.
r=6('A'): freq[A]=2, maxFreq=3. window=BABBA(5). 5-3=2>1 INVALID
    → freq[B]--, left=3. freq[B]=2. window=ABBA(4). 4-3=1≤1 valid.
```

maxLen = 4 

Example C — Fruit Into Baskets / Longest At Most K Distinct (Striver L5/L6)

Problem: You have two baskets (k=2). Each basket holds one type of fruit. Find longest contiguous subarray with at most 2 distinct values.

```
fruits = [1, 2, 1, 2, 3]
Answer: 4 (subarray [1,2,1,2])
```

java


```

class Solution {
    public int totalFruit(int[] fruits) {
        Map<Integer, Integer> basket = new HashMap<>(); // fruit type → count
        int left = 0, maxLen = 0;

        for (int right = 0; right < fruits.length; right++) {
            // EXPAND: add fruit to basket
            basket.put(fruits[right], basket.getOrDefault(fruits[right], 0) + 1);

            // SHRINK: while more than 2 types, remove from left
            while (basket.size() > 2) {
                int leftFruit = fruits[left];
                basket.put(leftFruit, basket.get(leftFruit) - 1);
                if (basket.get(leftFruit) == 0) basket.remove(leftFruit);
                left++;
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
// General version: replace 2 with k for "at most k distinct characters"
// Time: O(n) | Space: O(k)

```

PATTERN 4 — Variable Window with Frequency Map (Minimise Length)

When to Use

- Find the **SHORTEST** substring/subarray satisfying a condition
- Condition involves: "contains all characters of target", "at least K of something"
- **Template B** — expand until valid, then shrink while valid

The "Have vs Need" Framework

For minimum window problems:

need[c] = how many of char c are required (from target)

have[c] = how many of char c are in current window

formed = how many DISTINCT chars satisfy have[c] >= need[c]

required = total distinct chars we need to satisfy

```
Window is VALID when: formed == required  
While valid: record answer, try to shrink from left.
```

Example A — Minimum Window Substring (Hard)

Problem: Find smallest window in `(s)` that contains all characters of `(t)`.

```
s = "ADOBECODEBANC", t = "ABC"  
Answer: "BANC"
```

```
java
```

```

class Solution {
    public String minWindow(String s, String t) {
        if (s.isEmpty() || t.isEmpty()) return "";

        Map<Character, Integer> need = new HashMap<>();
        for (char c : t.toCharArray())
            need.put(c, need.getOrDefault(c, 0) + 1);

        int required = need.size(); // distinct chars we must satisfy
        int formed = 0;             // distinct chars currently satisfied

        Map<Character, Integer> have = new HashMap<>();
        int left = 0;
        int minLen = Integer.MAX_VALUE;
        int resLeft = 0, resRight = 0;

        for (int right = 0; right < s.length(); right++) {
            // EXPAND: add s[right]
            char c = s.charAt(right);
            have.put(c, have.getOrDefault(c, 0) + 1);

            // Check if this char's requirement is now satisfied
            if (need.containsKey(c) && have.get(c).equals(need.get(c)))
                formed++;

            // SHRINK: while window is valid (all chars satisfied), try to shrink
            while (formed == required) {
                // Update answer
                if (right - left + 1 < minLen) {
                    minLen = right - left + 1;
                    resLeft = left;
                    resRight = right;
                }

                // Remove s[left] from window
                char l = s.charAt(left);
                have.put(l, have.get(l) - 1);
                if (need.containsKey(l) && have.get(l) < need.get(l))
                    formed--; // no longer satisfying this char's requirement

                left++;
            }
        }

        return minLen == Integer.MAX_VALUE ? "" : s.substring(resLeft, resRight + 1);
    }
}

```

```
    }  
}  
// Time:  $O(|s| + |t|)$  | Space:  $O(|s| + |t|)$ 
```

Step-by-step trace for $s=\text{"ADOBECODEBANC"}$, $t=\text{"ABC"}$:

need={A:1, B:1, C:1}, required=3

right=0('A'): have={A:1}. A satisfied→formed=1. formed≠3, no shrink.

right=1('D'): have={A:1,D:1}. formed=1.

right=2('O'): formed=1.

right=3('B'): have+B=1. B satisfied→formed=2.

right=4('E'): formed=2.

right=5('C'): have+C=1. C satisfied→formed=3. ← VALID!

Shrink: window=ADOBEC(0..5), len=6. minLen=6, res=[0,5]

Remove A(left=0): have[A]=0 < need[A]=1 → formed=2. Stop shrinking.

left=1

right=6('O'): formed=2.

right=7('D'): formed=2.

right=8('E'): formed=2.

right=9('B'): have[B]=2. formed=2 (already had 1, now 2, still satisfied?

need=1, have=2≥1 but formed only increments when have==need exactly)

→ Wait: formed increments only when have[c] == need[c]. have[B] goes 1→2, need[B]=1, 2≠1 so no new formed.

right=10('A'): have[A]=1. A satisfied→formed=3. ← VALID!

Shrink: window=DOBECODEBA(1..10), len=10. minLen stays 6.

Remove D(left=1): formed still 3. left=2.

Shrink: window=OBECODEBA(2..10), len=9. minLen stays 6.

Remove O: formed still 3. left=3.

... keep shrinking until a needed char is removed.

Remove B(left=3 was 'B'? no...): let me think...

Actually left moves: D,O,B,E,C,O,D,E,B,A

When left hits 'C' at index 5: have[C] drops to 0 < need=1 → formed=2.

Stop.

left=6. Window=ODEBA(6..10). Before stopping: window was CODEBA, len=6 still.

right=11('N'): formed=2.

right=12('C'): have[C]=1. formed=3. ← VALID!

Shrink: window=ODEBANC(6..12), len=7. No improvement.

Remove O: formed=3, left=7.

Shrink: window=DEBANC(7..12), len=6. No improvement.

Remove D: left=8.

Shrink: window=EBANC(8..12), len=5. No improvement.

Remove E: left=9.

Shrink: window=BANC(9..12), len=4. minLen=4! resLeft=9, resRight=12.

Remove B: have[B]=0 < need=1 → formed=2. Stop. left=10.

Return s[9..12] = "BANC" 

Example B — Sliding Window for "Number of Substrings Containing All 3 Characters" (Striver L7)

Problem: Count substrings containing at least one 'a', one 'b', one 'c'.

Key Insight — Count from valid window:

When window [left..right] is valid (has a,b,c):

ALL substrings starting at left, left-1, ..., 0 and ending at right are also valid.

→ add (left + 1) to count for each valid right position.

Then shrink left by 1 to find minimal valid window (for next right).

java

```
class Solution {
    public int numberOfSubstrings(String s) {
        int[] count = new int[3]; // count of 'a','b','c'
        int left = 0, result = 0;

        for (int right = 0; right < s.length(); right++) {
            count[s.charAt(right) - 'a']++;

            // Shrink while valid (count of all 3 chars ≥ 1)
            while (count[0] > 0 && count[1] > 0 && count[2] > 0) {
                result += s.length() - right; // all extensions to the right
                count[s.charAt(left) - 'a']--;
                left++;
            }
        }
        return result;
    }
}

// Time: O(n) | Space: O(1)
```

PATTERN 5 — Monotonic Deque (Window Maximum)

When to Use

- Find the **maximum** (or minimum) in every window of size K

- Brute force is $O(nk)$ — need $O(n)$
- Keywords: "sliding window maximum/minimum"

The Core Idea — Monotonic Deque

A deque (double-ended queue) that stores INDICES of "useful" elements.
 "Useful" = could be the maximum of some future window.

INVARIANT: deque always stores indices in DECREASING order of their VALUES.
 Front of deque = index of the MAXIMUM in current window.
 Back of deque = index of the smallest "potentially useful" element.

When adding a new element at right:

1. Remove from BACK: any index whose value $\leq$ current value (they can never be max)
2. Add current index to BACK.
3. Remove from FRONT: if front index $<$ left (out of window).
4. Front of deque = current window maximum.

Visual:

nums = [3, 1, 3, -1, -3, 5, 3, 6, 7], k=3

```

right=0(3): deque=[0]          (values: [3])
right=1(1): 1<3, add. deque=[0,1] (values: [3,1])
right=2(3): remove 1( $\leq 3$ ), remove 0? 3 not < 3. deque=[0,2] (values:[3,3])
             window [0..2]: max=nums[deque[0]]=nums[0]=3
right=3(-1): add. deque=[0,2,3]  $\rightarrow$  but 0 < left=1? left=1 now.
             remove front 0 (index < left=1): deque=[2,3] (values:[3,-1])
             max=nums[2]=3
right=4(-3): add. deque=[2,3,4] (values:[3,-1,-3]), left=2.
             front=2, in window. max=3
right=5(5): remove all: 4(-3<5), 3(-1<5), 2(3<5). deque=[5], left=3.
             front=5  $\geq$  left=3. max=5
right=6(3): 3<5, add. deque=[5,6], left=4. max=nums[5]=5
right=7(6): remove 6(3<6), 5(5<6). deque=[7], left=5. max=6
right=8(7): remove 7(6<7). deque=[8], left=6. max=7
  
```

Window maxes: [3, 3, 3, 5, 5, 6, 7] 

Example — Sliding Window Maximum ★ (Hard)

java

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque<Integer> deque = new ArrayDeque<>(); // stores INDICES
        int left = 0;

        for (int right = 0; right < n; right++) {
            // REMOVE from back: indices whose values are ≤ current (useless)
            while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[right])
                deque.pollLast();

            // ADD current index to back
            deque.offerLast(right);

            // REMOVE from front: if front index is outside current window
            if (deque.peekFirst() < left) {
                deque.pollFirst();
            }

            // RECORD max when we have a full window
            if (right >= k - 1) {
                result[left] = nums[deque.peekFirst()]; // front = max
                left++;
            }
        }
        return result;
    }
}

// Time: O(n) — each element added and removed from deque at most once
// Space: O(k) — deque holds at most k elements
```

Why is it $O(n)$ and not $O(nk)$?

Each element enters the deque ONCE and leaves the deque ONCE.

Total operations = $2n = O(n)$.

The while loop might run multiple times per step, but TOTAL across all steps = $O(n)$.

How to Identify the Pattern

READ the problem. Ask these questions in order:

START

|



Are the two endpoints of interest at OPPOSITE ends, moving toward each other?

OR is the array sorted and you need pairs/triplets summing to a value?

|— YES → PATTERN 1 (Opposite Ends Two Pointers)

| Palindrome: compare chars, skip non-alphanumeric

| Two Sum II: $\text{sum} < \text{target} \rightarrow \text{left}++$, $\text{sum} > \text{target} \rightarrow \text{right}--$

| 3Sum: fix i, two pointers on rest, skip duplicates

| Container: move shorter wall inward

| Trap Water: process side with smaller maxHeight

|— NO ↓

Is the window size K FIXED and given explicitly?

OR does the problem compare a window to a PATTERN of known length?

|— YES → PATTERN 2 (Fixed Size Sliding Window)

| Slide a window of size k, add right, remove left when size > k

| Permutation check: freq array comparison

|— NO ↓

Are you finding the MINIMUM window / SHORTEST substring satisfying a condition?

(contains all chars, $\text{sum} \geq K$, etc.)

|— YES → PATTERN 4 (Variable Window – Minimise, Template B)

| Expand until valid. Shrink while still valid. Answer INSIDE while.

| Track: need[], have[], formed, required.

|— NO ↓

Is the problem asking for MAX in every window of size K?

|— YES → PATTERN 5 (Monotonic Deque)

| Deque stores indices in decreasing value order.

| Front = max of current window.

|— NO ↓

Default → PATTERN 3 (Variable Window – Maximise, Template A)

Longest substring/subarray satisfying a condition.

Expand right always. Shrink left when INVALID.

Answer OUTSIDE while. State = freq map or counter.
Invalid when: duplicates, > k distinct, replacements > k.

Problem → Pattern Mapping

Problem	Pattern	Key Trick
Valid Palindrome	Pattern 1	Skip non-alphanumeric, compare from both ends
Two Sum II (Sorted)	Pattern 1	sum<target→left++, sum>target→right--
3Sum	Pattern 1	Sort + fix i + two pointers, skip duplicates
Container With Most Water	Pattern 1	Move the SHORTER wall inward
Trapping Rain Water	Pattern 1	Process side with smaller maxHeight, track maxL/maxR
Permutation in String	Pattern 2	Fixed window size = len(s1), compare freq arrays
Maximum Points from Cards	Pattern 2	Min window of size (n-k) = totalSum - minWindow
Longest Substring No Repeat	Pattern 3	lastIndex map, jump left past duplicate
Longest Repeating Char Replacement	Pattern 3	Invalid: (windowSize - maxFreq) > k
Fruit Into Baskets / K Distinct	Pattern 3	Map of counts, invalid when map.size() > k
Minimum Window Substring	Pattern 4	need/have/formed/required framework
# Substrings with All 3 Chars	Pattern 4	Shrink while valid, add (n-right) per valid position
Sliding Window Maximum	Pattern 5	Monotonic deque (indices), front = max

Complexity Cheat Sheet

Problem	Time	Space	Notes
Valid Palindrome	$O(n)$	$O(1)$	Two pointers, skip non-alnum
Two Sum II	$O(n)$	$O(1)$	Sorted $\rightarrow$ two pointers
3Sum	$O(n^2)$	$O(1)$	Sort + fix + two pointers
Container With Most Water	$O(n)$	$O(1)$	Move shorter wall
Trapping Rain Water	$O(n)$	$O(1)$	Two pointers with maxL/maxR
Permutation in String	$O(n)$	$O(1)$	Fixed window, freq compare
Longest Substring No Repeat	$O(n)$	$O(n)$	lastIndex HashMap
Longest Repeating Char Replace	$O(n)$	$O(1)$	freq[26], maxFreq trick
Fruit Into Baskets	$O(n)$	$O(k)$	HashMap, shrink when $> k$ types
Minimum Window Substring	$O(n+m)$	$O(n+m)$	need/have maps
Sliding Window Maximum	$O(n)$	$O(k)$	Monotonic deque

Common Mistakes to Avoid

✗ Mistake 1: 3Sum — not sorting first

```
java
// WRONG - two pointers require sorted array; also can't skip duplicates reliably
int left = i + 1, right = nums.length - 1;
// Without sorting, left/right movement logic breaks

// CORRECT - always sort first
Arrays.sort(nums);
```

✗ Mistake 2: 3Sum — skipping duplicates in wrong place

```
java
```

```
// WRONG – skipping BEFORE checking means you miss valid triplets
for (int i = 0; i < nums.length; i++) {
    if (nums[i] == nums[i-1]) continue; // ← ArrayIndexOutOfBoundsException at i=0!

// CORRECT – check i > 0 before comparing
    if (i > 0 && nums[i] == nums[i-1]) continue;
```

✗ Mistake 3: Variable window — updating answer INSIDE shrink loop (Template A)

```
java

// WRONG for Template A (maximise) – you shrink the window then record it
while (invalid) {
    left++;
    maxLen = Math.max(maxLen, right - left + 1); // wrong position!
}

// CORRECT for Template A – update AFTER the while loop (when window is valid)
while (invalid) {
    left++;
}
maxLen = Math.max(maxLen, right - left + 1); // ← outside the while
```

✗ Mistake 4: Minimum Window — updating answer OUTSIDE shrink loop (Template B)

```
java

// WRONG for Template B (minimise) – you want the answer WHILE still valid
// Updating outside means you've already made the window invalid
while (formed == required) {
    left++;
}
minLen = Math.min(minLen, right - left + 1); // window is now invalid!

// CORRECT for Template B – update INSIDE the while (when window is valid)
while (formed == required) {
    minLen = Math.min(minLen, right - left + 1); // ← inside!
    left++;
}
```

✗ Mistake 5: Trapping Rain Water — not updating maxLeft/maxRight before computing water

java

```
// WRONG order - checking water before updating max
totalWater += maxLeft - height[left]; // maxLeft might be stale!
maxLeft = Math.max(maxLeft, height[left]);
left++;

// CORRECT - update max FIRST, then compute water (or combine with Math.max)
if (height[left] >= maxLeft) {
    maxLeft = height[left]; // no water here, this is a wall
} else {
    totalWater += maxLeft - height[left]; // water here
}
left++;
```

✗ Mistake 6: Sliding Window Maximum — removing from front before adding new element

java

```
// WRONG - should add THEN check front, otherwise might check stale front
deque.pollFirst(); // remove out-of-window element
deque.offerLast(right); // add new

// CORRECT order: remove old from back → add new → remove stale from front →
while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[right]) deque.pollLast();
deque.offerLast(right);
if (deque.peekFirst() < left) deque.pollFirst();
if (right >= k - 1) result[left++] = nums[deque.peekFirst()];
```

✗ Mistake 7: Longest Repeating Char Replacement — decreasing maxFreq on shrink

java

```
// WRONG – recalculating maxFreq after shrink causes O(26n) and is unnecessary
while ((right - left + 1) - maxFreq > k) {
    freq[s.charAt(left) - 'A']--;
    maxFreq = 0;
    for (int f : freq) maxFreq = Math.max(maxFreq, f); // O(26) per step
    left++;
}

// CORRECT – never decrease maxFreq. Stale maxFreq only prevents expansion,
// which is fine – we only care about windows LONGER than our current best.
while ((right - left + 1) - maxFreq > k) {
    freq[s.charAt(left) - 'A']--;
    left++;
    // maxFreq stays – intentional!
}
```

Quick Reference Card

TWO POINTERS + SLIDING WINDOW QUICK CARD	
PATTERN 1	Opposite ends: left=0, right=n-1
Opposite Ends	sum<target → left++
	sum>target → right--
	3Sum: sort + fix i + two ptrs + skip dups
	Trap Water: process smaller maxHeight side
PATTERN 2	Fixed size k: slide, add right, remove left
Fixed Window	Permutation: compare freq[26] arrays
	Max Cards: min window of size (n-k)
PATTERN 3	TEMPLATE A (maximise window)
Variable Max	expand right always
	while (INVALID): remove left, left++
	answer = OUTSIDE while
	No repeat: jump left past lastIndex
	Char replace: invalid=(size-maxFreq)>k
	K distinct: invalid=map.size()>k
PATTERN 4	TEMPLATE B (minimise window)
Variable Min	expand right until valid

	while (VALID): answer INSIDE, shrink left Min Window: need/have/formed/required formed++ only when have[c]==need[c] exactly
PATTERN 5 Window Maximum	Monotonic deque of INDICES Remove from back if $\text{nums}[\text{back}] \leq \text{nums}[\text{right}]$ Add right to back Remove from front if out of window Front = current window max
EXACTLY K TRICK	$\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k-1)$ atMost: shrink when count > k result += right - left + 1 per step

Core Building Blocks

java

```

// — 1. OPPOSITE ENDS TWO POINTER —————
int left = 0, right = n - 1;
while (left < right) {
    if (condition) { /* process */ left++; right--; }
    else if (needMore) left++;
    else right--;
}

// — 2. FIXED WINDOW —————
for (int right = 0; right < n; right++) {
    // add arr[right]
    if (right - left + 1 > k) {
        // remove arr[left]
        left++;
    }
    if (right - left + 1 == k) { /* record answer */ }
}

// — 3. TEMPLATE A – VARIABLE WINDOW (MAXIMISE) —————
int left = 0, maxLen = 0;
for (int right = 0; right < n; right++) {
    // add arr[right] to window state
    while (/* invalid */) {
        // remove arr[left] from state
        left++;
    }
    maxLen = Math.max(maxLen, right - left + 1); // OUTSIDE while
}

// — 4. TEMPLATE B – VARIABLE WINDOW (MINIMISE) —————
int left = 0, minLen = Integer.MAX_VALUE;
for (int right = 0; right < n; right++) {
    // add arr[right] to window state
    while (/* valid */) {
        minLen = Math.min(minLen, right - left + 1); // INSIDE while
        // remove arr[left] from state
        left++;
    }
}

// — 5. MONOTONIC DEQUE (MAX WINDOW) —————
Deque<Integer> dq = new ArrayDeque<>(); // stores indices
for (int right = 0; right < n; right++) {
    while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[right]) dq.pollLast()
}

```



```

dq.offerLast(right);
if (dq.peekFirst() < left) dq.pollFirst();
if (right >= k - 1) { result[left] = nums[dq.peekFirst()]; left++; }
}

// — 6. EXACTLY K = AT MOST K - AT MOST K-1 —————
int atMost(int[] a, int k) {
    int left = 0, count = 0, res = 0;
    for (int right = 0; right < a.length; right++) {
        if (qualifies(a[right])) count++;
        while (count > k) { if (qualifies(a[left])) count--; left++; }
        res += right - left + 1;
    }
    return res;
}
// exactlyK(a, k) = atMost(a, k) - atMost(a, k-1)

```

Notes Version 1.0 — Covers all 5 NeetCode Two Pointers + all 6 NeetCode Sliding Window problems + Striver's Two Pointers & Sliding Window series (L1-L12) using 5 patterns. Revise order: Pattern 1 (Opposite Ends) → Pattern 3 (Template A) → Pattern 4 (Template B) → Pattern 2 (Fixed) → Pattern 5 (Deque)